

W1seGuy - CTF

[Kaynak](#)

Hedef Analizi

Bu doküman, "W1seGuy" isimli kriptografi (Cryptography) odaklı CTF (Capture The Flag) makinesinin derinlemesine teknik analizini ve çözüm adımlarını içermektedir. Makine, zayıf yapılandırılmış bir özel şifreleme algoritmasının, bilinen açık metin saldırısı (Known Plaintext Attack) yöntemleriyle nasıl istismar edileceğini (Exploitation) temel almaktadır.

1. Durum Analizi

Bağlantıyı kurduğumuzda sistemin bize ne verdiğine ve bizden ne istediğine bakıyoruz:

```
netcat 10.113.147.5 1337
# Çıktı: This XOR encoded text has flag 1: 6c38054a17091124...
# Çıktı: What is the encryption key?
```

- **Bize verilen:** Hexadecimal formatta şifrelenmiş bir metin (Ciphertext).
- **Bizden istenen:** Bu metni şifrelemek için kullanılan 5 karakterli anahtar (Key).
- **Kritik Kural:** Kaynak koddan biliyoruz ki, şifreleme anahtarı 5 karakter uzunluğunda ve her Netcat bağlantısında (her oturumda) rastgele yeniden üretiliyor.

2. Zafiyetin Mantığı: Known Plaintext Attack (KPA)

XOR şifrelemesinin altın kuralı şudur:

Şifreli Metin $\oplus$ Açık Metin = Anahtar

TryHackMe'de tüm bayrakların THM{ ile başladığını biliyoruz. Yani elimizde 4 karakterlik kesin bir "Açık Metin" (Plaintext) var.

3. Flag 1'i Bulmak

Sisteme bağlandın ve sana bir şifreli metin verdi. Anahtar 5 karakter ama elimizde sadece THM{ (4 karakter) var. 5. karakteri bilmiyoruz. İşte Flag 1'i bulmak için burada küçük bir **Brute Force (Kaba Kuvvet)** devreye giriyor.

Mantık şu:

1. İlk 4 byte'ı THM{ ile XOR'la -> Anahtarın ilk 4 karakterini bul (Örn: UDC0).
2. Anahtarın 5. karakteri İngilizce alfanümerik bir harf veya sayıdır. Kalan 62 ihtimali (A-Z, a-z, 0-9) tek tek dene.

3. Hangi harfi denediğinde anlamlı bir İngilizce cümle/bayrak ortaya çıkıyorsa, doğru anahtar olur.

Bunu yapan Python kodu şudur (Flag 1'i böyle elde ederiz):

```
import string

# Hexadecimal Ciphertext (Netcat'ten aldığımız veri)
hex_cipher =
"6c38054a170911245f137d083c70134c442b5a04791e3a0206543c3159324a043101124a080
7431a"
cipher_bytes = bytes.fromhex(hex_cipher)

# 1. İlk 4 karakteri KPA (Known Plaintext Attack) ile buluyoruz
key_prefix = bytes([cipher_bytes[i] ^ b"THM{"[i] for i in range(4)])

# 2. 5. Karakter için Brute-Force işlemi
for char in string.ascii_letters + string.digits:
    # Anahtarı oluştur (Örn: 8pH1 + a)
    test_key = key_prefix + char.encode()

    # Tüm metni bu test anahtarıyla çöz
    decrypted = bytes([cipher_bytes[i] ^ test_key[i % 5] for i in
range(len(cipher_bytes))])

    # AKILLI FİLTRELEME: Metin THM{ ile başlayıp } ile BİTİYOR MU?
    if decrypted.startswith(b"THM{") and decrypted.endswith(b"}"):
        print("[+] Hedef Doğrulandı. Çöp veriler elendi.")
        print(f"[*] Kusursuz Anahtar: {test_key.decode()}")
        print(f"[*] Flag 1: {decrypted.decode()}")
        break # Doğru sonucu bulduk, döngüyü kır ve işlemi bitir!
```

Birinci bayrağı bu şekilde elde ediyoruz..

```
PS C:\Users\Gökhan> & C:/Users/Gökhan/AppData/Local/Programs/Python/Python313/python.exe c:/Users/Gökhan/Desktop/source-1705339805281.py
[+] Hedef Doğrulandı. Çöp veriler elendi.
[*] Kusursuz Anahtar: 8pH1g
[*] Flag 1: THM{p1a1nT}
PS C:\Users\Gökhan>
```

4. Flag 2'yi Almak

Önceki adımda yazdığımız optimize edilmiş Python scripti (Akıllı Filtreleme), aslında tek taşla iki kuş vurmamızı sağlar. Script, hem hex dizisinin içine gizlenmiş **Flag 1**'i çözer hem de o anki oturuma ait **şifreleme anahtarını (Key)** anında hesaplar. Bu nedenle, işi uzatmaya veya ikinci bir matematiksel işlem yapmaya hiç gerek yoktur.

Anahtarı **What is the encryption key?** sorusuna cevap olarak yazdığınızda, sunucu gönderdiğiniz anahtarı doğrulayacak ve `flag.txt` içerisinde okuduğu nihai bayrağı (Flag 2) ekrana basarak bağlantıyı başarıyla sonlandıracaktır:

```
root@ip-10-113-108-53:~# netcat 10.113.147.5 1337
This XOR encoded text has flag 1: 6c38054a170911245f137d083c70134c442b5a04791e3a
0206543c3159324a043101124a0807431a
What is the encryption key? 8pH1g
Congrats! That is the correct key! Here is flag 2: THM{[REDACTED]}
FuN_n0?}
█
```